

CS5224 Lab 1: The Wedding Photographer's Cloud

Student ID A033030X · AWS · ap-southeast-1 (Singapore) · 18 September 2026

Submission details

Item	Value
Student ID	A033030X
Cloud provider	Amazon Web Services (AWS), used for the whole lab
Region	ap-southeast-1 (Asia Pacific, Singapore)
Bucket	priya-weddings-a033030x
Wedding prefix	tan-and-mei-wedding-2026/

Assumptions

- S3 bucket names allow only lowercase letters, digits, dots and hyphens, so student ID A033030X appears in lowercase in the bucket name.
- Singapore was chosen because Priya and her clients are in Singapore, which gives the lowest latency and keeps data in-country.
- Sample images are 5 small PNG placeholders (47,319 bytes each), not real wedding photos.
- Storage figures use decimal units (1 GB = 1,000 MB) unless marked GiB.

Part 1: Build one client's gallery

Anyone with an image's URL can view that image, but no one can list the bucket. The bucket was defined with AWS CDK (TypeScript) and deployed as the CloudFormation stack `PriyaWeddingsPoc`. Five images were then uploaded with the AWS CLI.

Configuration

Setting	Value	Why
Object Ownership	Bucket owner enforced (ACLs disabled)	Access is controlled only by the bucket policy
Block public ACLs / Ignore public ACLs	On	Objects can't be made public through ACLs
Block public bucket policies / Restrict public buckets	Off	Needed so the public-read policy below takes effect
Default encryption	SSE-S3 (AES-256)	Objects are encrypted at rest
Objects	photo-01.png to photo-05.png under tan-and-mei-wedding-2026/	3-5 sample images, as the spec requires

Bucket policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": { "AWS": "*" },
```

```

    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::priya-weddings-a033 030x",
      "arn:aws:s3:::priya-weddings-a033 030x/*"
    ],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  },
  {
    "Sid": "PublicReadWeddingPhotos",
    "Effect": "Allow",
    "Principal": { "AWS": "*" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::priya-weddings-a033 030x/tan-and-mei-wedding-2026/*"
  }
]
}

```

The policy grants `s3:GetObject` only on objects under the wedding prefix. It grants no `s3:ListBucket`, so the bucket can't be browsed. The first statement rejects any request made without HTTPS.

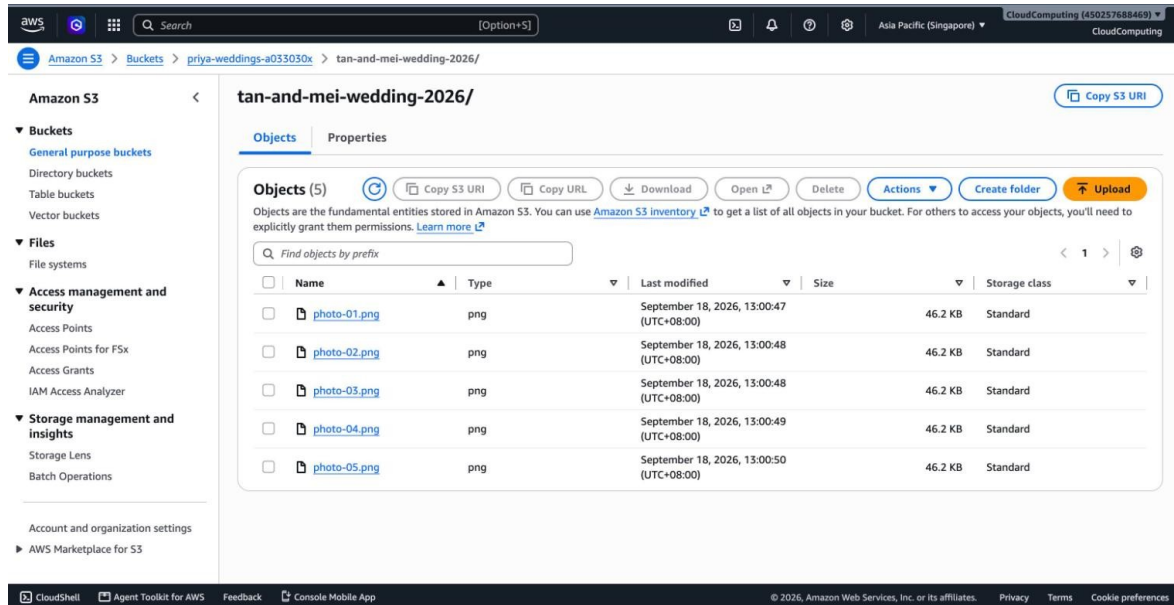
Verification (anonymous requests with `curl`, 18 Sep 2026)

Request	Result	Meaning
GET https://.../tan-and-mei-wedding-2026/photo-01.png	200, image/png	Public object access works
GET https://priya-weddings-a033 030x.s3.ap-southeast-1.amazonaws.com/	403 AccessDenied	Bucket listing is blocked
GET .../?list-type=2&prefix=tan-and-mei-wedding-2026/	403	Listing the prefix is blocked too
GET http://.../photo-01.png (no TLS)	403	HTTPS is enforced

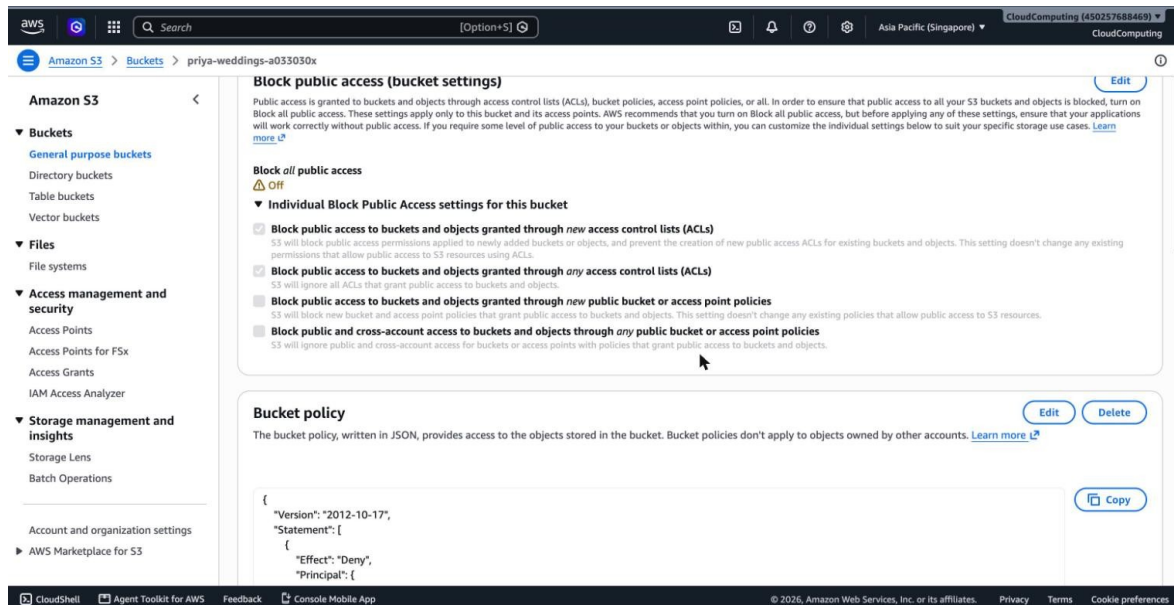
Without `s3:ListBucket`, S3 also returns 403 rather than 404 for a key that doesn't exist. Outsiders therefore can't probe which file names exist.

Part 1 evidence: screenshots

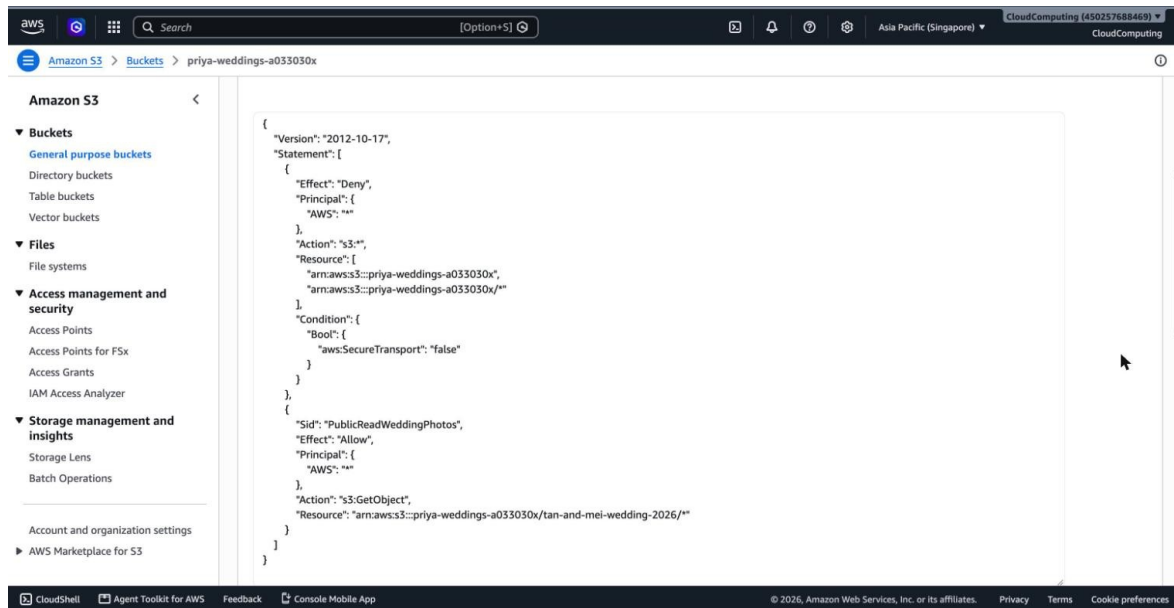
Screenshot 1 — Bucket contents. The tan-and-mei-wedding-2026/ prefix holds photo-01.png to photo-05.png, each 46.2 KB, last modified 18 Sep 2026 13:00 (UTC+08:00), storage class Standard.



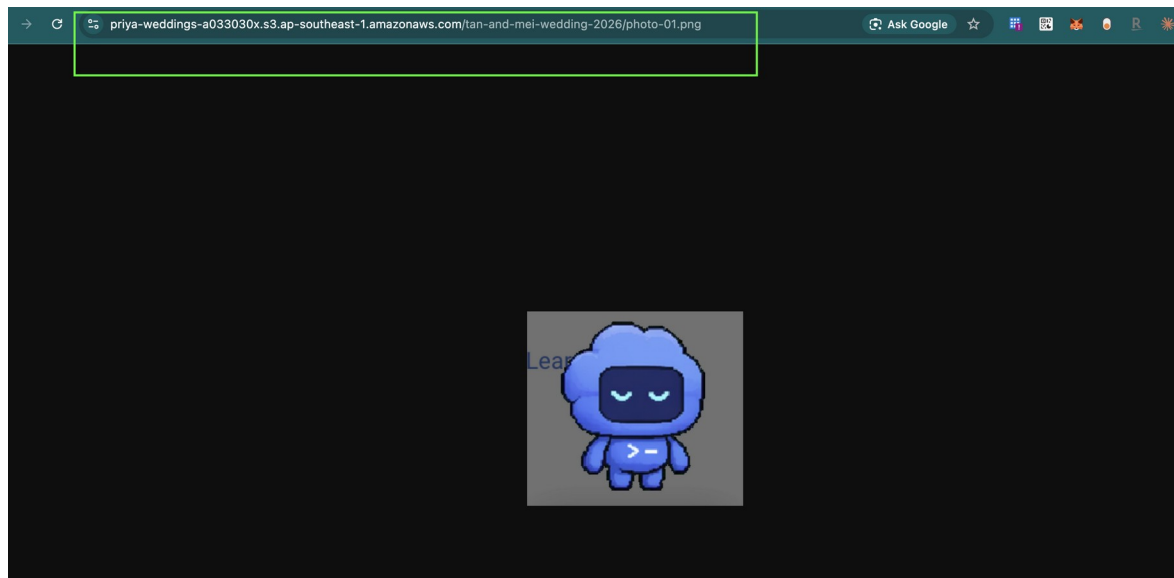
Screenshot 2a — Block Public Access. The two ACL blocks are on; the two policy blocks are off, so the bucket policy can grant public reads.



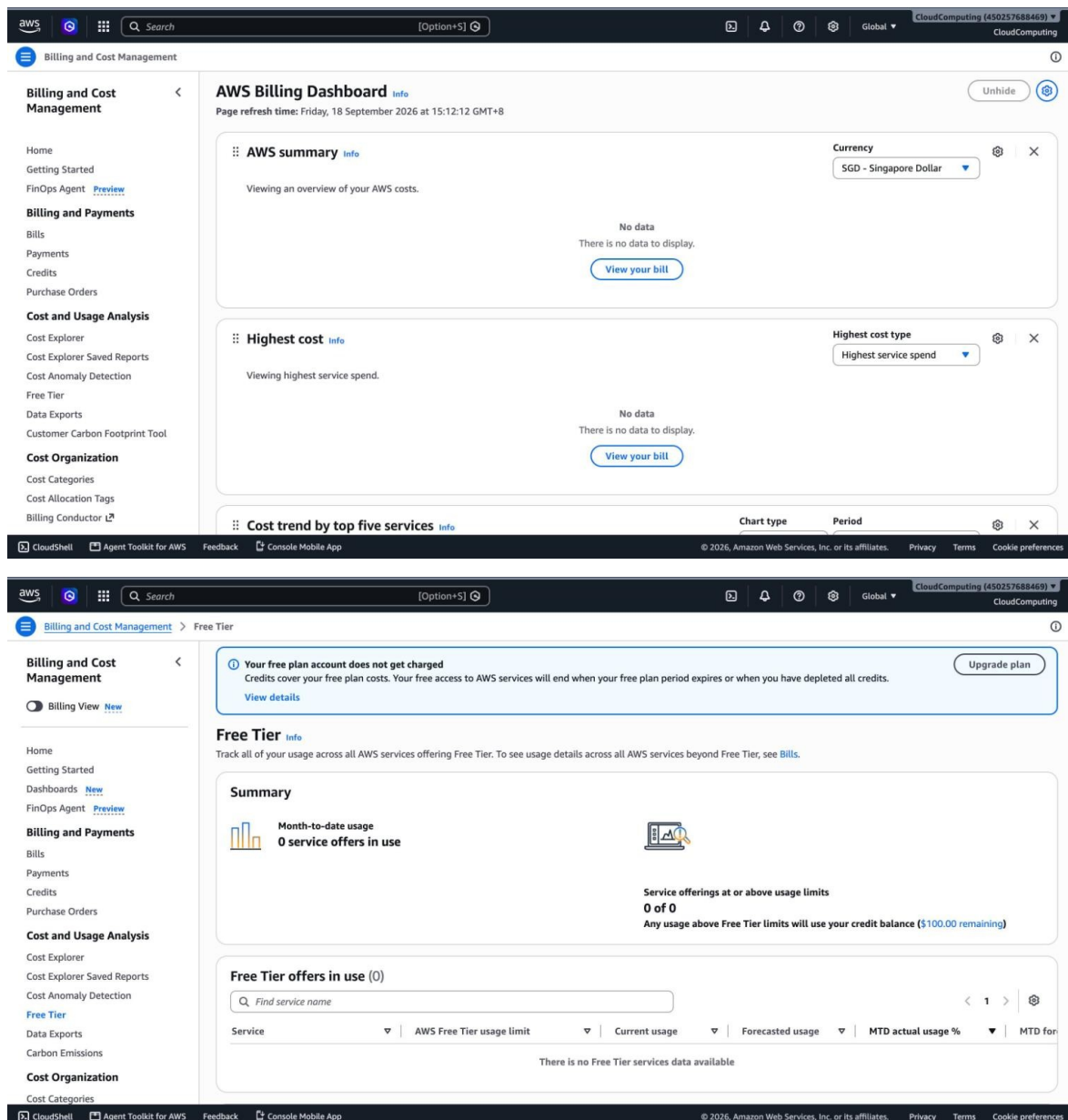
Screenshot 2b — Bucket policy. HTTPS-only deny, plus s3:GetObject for everyone on tan-and-mei-wedding-2026/* only. No s3:ListBucket is granted.



Screenshot 3 — Image loading through its S3 URL. Chrome's address bar shows `priya-weddings-a033030x.s3.ap-southeast-1.amazonaws.com/tan-and-me-wedding-2026/photo-01.png`, opened without signing in.



Screenshot 4 — Billing dashboard and Free Tier. No cost data yet on 18 Sep 2026; the account is on the AWS free plan with \$100.00 of credit remaining.



Part 1 reflection: annual storage

Calculation (decimal units, 1 GB = 1,000 MB)

$40 \text{ weddings/year} \times 400 \text{ photos/wedding} \times 5 \text{ MB/photo} = 80,000 \text{ MB} = \mathbf{80 \text{ GB per year}}$ ($\approx 74.5 \text{ GiB}$ in binary units, since $80 \times 10^9 \div 2^{30} \approx 74.5$).

Answer

Priya needs about 80 GB of new storage each year (decimal units), and the total grows by that amount every year. This would not stay within the free tier: my account is on AWS's credit-based free plan (\$100 credit, time-limited), which has no permanent free S3 storage, and even the older 5 GB S3 allowance would be exceeded by about the third wedding. At the S3 Standard price in ap-southeast-1 of \$0.025 per GB-month, one year of photos costs about $80 \times \$0.025 = \mathbf{\$2.00 \text{ per month}}$, so credits would absorb it only temporarily and the cost keeps growing unless old galleries move to a cheaper tier.

Price source: AWS Price List API for AmazonS3, region ap-southeast-1, queried 18 Sep 2026 (S3 Standard, first 50 TB: \$0.025 per GB-month).

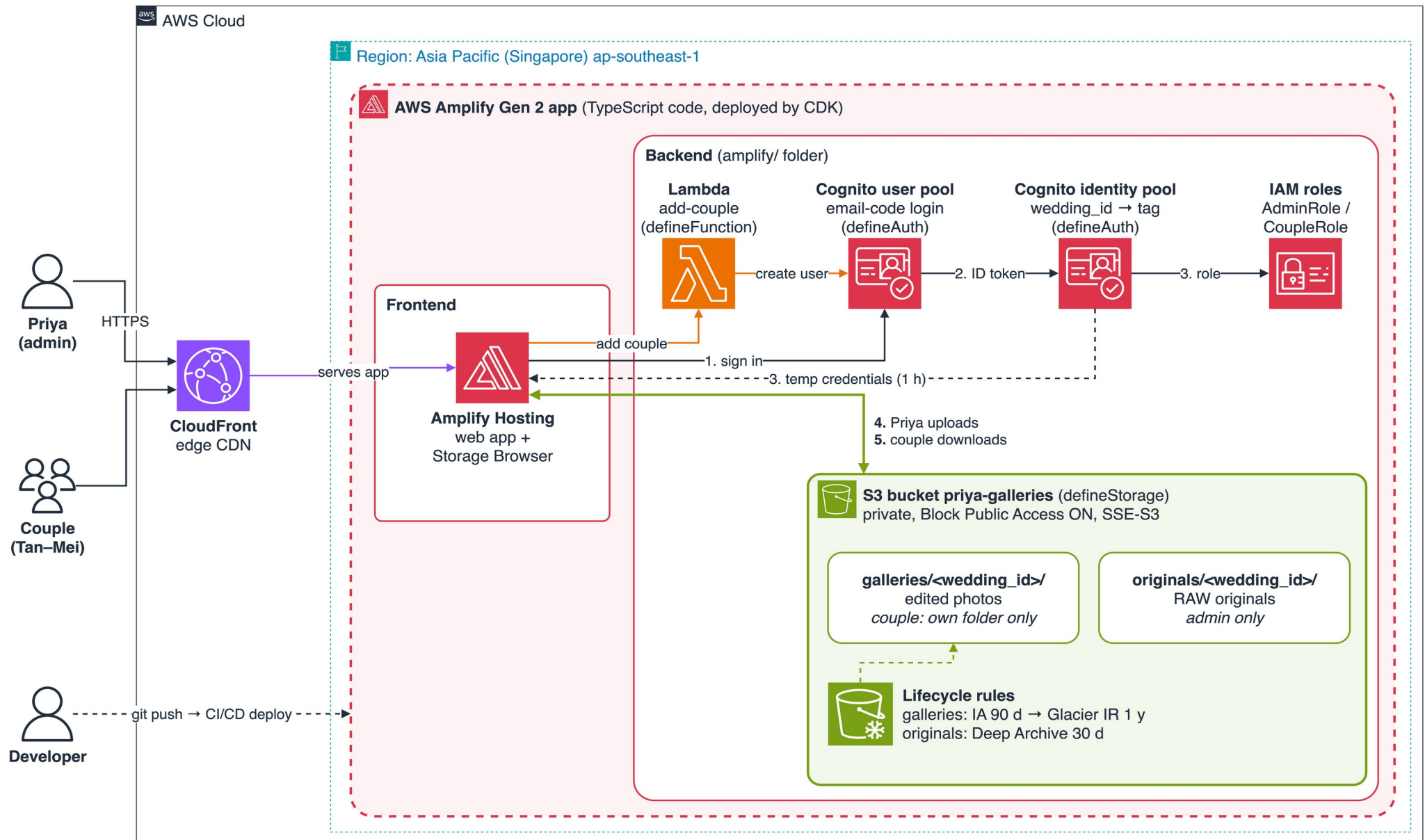
Part 2: Priya's full system

Priya's system is a private S3 bucket behind Cognito sign-in. The only custom code is one small "add couple" function; every other piece is a managed AWS service.

a) Architecture diagram

The system is one AWS Amplify Gen 2 app in ap-southeast-1: Amplify Hosting serves the web app through CloudFront, and the backend (Cognito, S3, Lambda) is defined as TypeScript code and deployed with AWS CDK. Each couple's sign-in carries a `wedding_id` tag, and one IAM policy lets that tag read only its own folder.

Priya's Wedding Gallery System (AWS Amplify Gen 2, ap-southeast-1)



Why couples can't see each other: Tan-Mei's temporary credentials carry wedding_id=tan-mei-2026, so CoupleRole only allows galleries/tan-mei-2026/*. Any other folder returns 403.

How to read it:

Flow	What happens
1	Priya and couples sign in on Cognito's hosted pages. Couples use a one-time email code, so they have no password.
2-3	The identity pool exchanges the ID token for temporary AWS credentials. It maps the user's <code>custom:wedding_id</code> attribute to a session tag and picks the role: <code>admin</code> group gets <code>AdminRole</code> , everyone else gets <code>CoupleRole</code> .
4	Priya uploads straight from her browser to S3 with <code>AdminRole</code> , which can write to <code>galleries/*</code> and <code>originals/*</code> .
5	A couple's browser lists and downloads from S3 directly. <code>CoupleRole</code> only allows <code>galleries/\${aws:PrincipalTag/wedding_id}/*</code> .
Add couple	Priya fills in a name, email and wedding ID; a small Lambda creates the Cognito user with that <code>wedding_id</code> and Cognito emails the invite.

Edited photos ("processed") live in `galleries/<wedding_id>/`. The camera originals live in `originals/<wedding_id>/`, which only `AdminRole` can reach.

b) Service selection table

Service	Provider	Purpose in Priya's system	Why this was chosen over alternatives
Storage: Amazon S3 (one private bucket, <code>galleries/</code> and <code>originals/</code> prefixes)	AWS	Stores every photo. Block Public Access is fully on and objects are encrypted with SSE-S3.	Unlimited capacity at \$0.025 per GB-month with 11 nines of durability. EBS or EFS would cost more and need a server in front. One bucket with prefixes keeps a single lifecycle and policy setup.
Access control: Amazon Cognito user pool (managed login)	AWS	Accounts for Priya and each couple. AWS hosts the sign-in pages; couples sign in with a one-time email code.	No login code to write or secure. Couples have no password to forget. It's free up to 10,000 monthly active users; Priya has about 80 couples a year.
Access control: Cognito identity pool + IAM (tag-based access)	AWS	Turns a sign-in into temporary AWS credentials tagged with <code>wedding_id</code> . One <code>CoupleRole</code> policy allows only <code>galleries/\${aws:PrincipalTag/wedding_id}/*</code> .	One policy covers every couple, so adding a wedding never changes IAM. S3 Access Grants would need an IAM Identity Center account per couple. Presigned links from a custom API would mean writing and securing that API.
Web app: AWS Amplify Gen 2: Hosting (on Amazon CloudFront) + Amplify UI Storage Browser	AWS	The site Priya and couples open. Storage Browser gives folder browsing, drag-and-drop upload and download, with no custom UI.	Priya uploads weekly without touching the AWS console. Hosting deploys from Git and serves through CloudFront over HTTPS. Gen 2 also defines

Service	Provider	Purpose in Priya's system	Why this was chosen over alternatives
			Cognito, S3 and Lambda as TypeScript code (defineAuth, defineStorage, defineFunction) deployed by CDK, so the whole system can be rebuilt from Git. A custom React gallery would look nicer but take weeks to build.
Upload helper: AWS Lambda add-couple	AWS	Priya's "add couple" form: creates the Cognito user with its wedding_id and triggers the invite email.	The only custom code, about 20 lines. Doing this by hand in the Cognito console is error-prone for a non-technical user.
Archival: S3 Lifecycle rules (Standard-IA, Glacier Instant Retrieval, Deep Archive)	AWS	Moves galleries to cheaper tiers as they age; originals go straight to Deep Archive.	Configuration only; nothing to run. Cuts year-5 storage cost by about 70% (see the cost estimate below).
Monitoring: AWS Budgets	AWS	Emails Priya if monthly spend passes \$10.	Guards against surprise bills without her checking the console.
<i>Rejected:</i> Pixieset / ShootProof (SaaS)	Third party	A ready-made client-gallery product.	About \$10–30 per month and polished, but Priya wouldn't own the storage, plans cap storage, and there's no cheap archive tier.
<i>Rejected:</i> AWS Transfer Family web apps	AWS	A no-code S3 browser portal.	\$0.50 per hour in Singapore (about \$365 per month), and every couple would need an IAM Identity Center account.

c) Design rationale

1. Keeping each gallery private. The S3 bucket is never public: Block Public Access is fully on and there is no public bucket policy, unlike the Part 1 proof of concept. The only way in is temporary credentials from the Cognito identity pool. A couple's credentials carry the session tag `wedding_id=tan-mei-2026`, and `CoupleRole` allows `s3:GetObject` only on `galleries/${aws:PrincipalTag/wedding_id}/*`, and `s3:ListBucket` only with that prefix. So when the Tan-Mei couple requests `galleries/wong-lim-2026/...`, IAM finds no matching allow and S3 returns 403. The tag comes from Cognito, not from the URL, so editing a link cannot change it. `originals/` is not in `CoupleRole` at all.

2. Identities, credentials and links. Priya and each couple have a Cognito user. Priya belongs to the `admin` group; each couple has a `custom:wedding_id` attribute set once by the `add-couple` Lambda. Couples sign in with a one-time email code, so no password is stored or shared. The identity pool issues credentials that expire after 1 hour, and Storage Browser uses them to sign each download, so any download link copied out of the app stops working within the hour. Nothing long-lived, such as an access key, is ever given to Priya or a couple.

3. Main tradeoff. I chose simplicity over customisation. Storage Browser is a file browser, not a photo grid with thumbnails and favourites, and a bespoke gallery app would give clients a nicer experience. In return there is almost no code to maintain, which suits a business with no engineer. A second tradeoff is in the

archive: old galleries go to Glacier Instant Retrieval (\$0.005 per GB-month) rather than Deep Archive (about \$0.002). That costs about 2.5× more per GB, but a couple can still open a five-year-old gallery instantly instead of waiting up to 12 hours for a restore. RAW originals, which are rarely needed, do go to Deep Archive.

4. If the business tripled. 120 weddings and 240 GB a year is still small for S3, and 10,000 Cognito users covers it with room to spare, so the core design stays. I would change three things. First, add thumbnails and a photo-grid page, with resized images generated and cached through CloudFront, because clients would expect a polished product. Second, if Priya hires photographers, add a `studio_id` tag alongside `wedding_id` so each photographer manages only their weddings. Third, add a "download all as ZIP" job (AWS Step Functions + Lambda) so couples don't download 400 files one by one. If uptime became critical, I would also turn on S3 Cross-Region Replication to a second region.

If a service fails. S3 stores data across at least three Availability Zones in the region. Cognito and Amplify Hosting are managed regional services. If Cognito were unavailable, couples could not sign in, but no photos would be lost or exposed.

Retention cost estimate

By year 5, lifecycle tiers bring gallery storage to about \$2.93 a month, versus \$10.00 if everything stayed in S3 Standard (about 71% less).

End of year	Galleries stored (GB)	All S3 Standard (USD/month)	With lifecycle tiers (USD/month)
1	80	2.00	1.33
2	160	4.00	1.73
3	240	6.00	2.13
4	320	8.00	2.53
5	400	10.00	2.93

Assumptions: ap-southeast-1 prices from the AWS Price List API (18 Sep 2026): S3 Standard \$0.025, Standard-IA \$0.0138, Glacier Instant Retrieval \$0.005 per GB-month. The current year's 80 GB is taken as 25% Standard and 75% Standard-IA (it moves at 90 days); every older year sits in Glacier Instant Retrieval. Lifecycle transition requests (about 16,000 objects a year) add only a few cents. RAW originals are excluded because their size is unknown.

Repository: <https://github.com/horiiiiiii032929/cs5224-lab1-priya-wedding-gallery>